

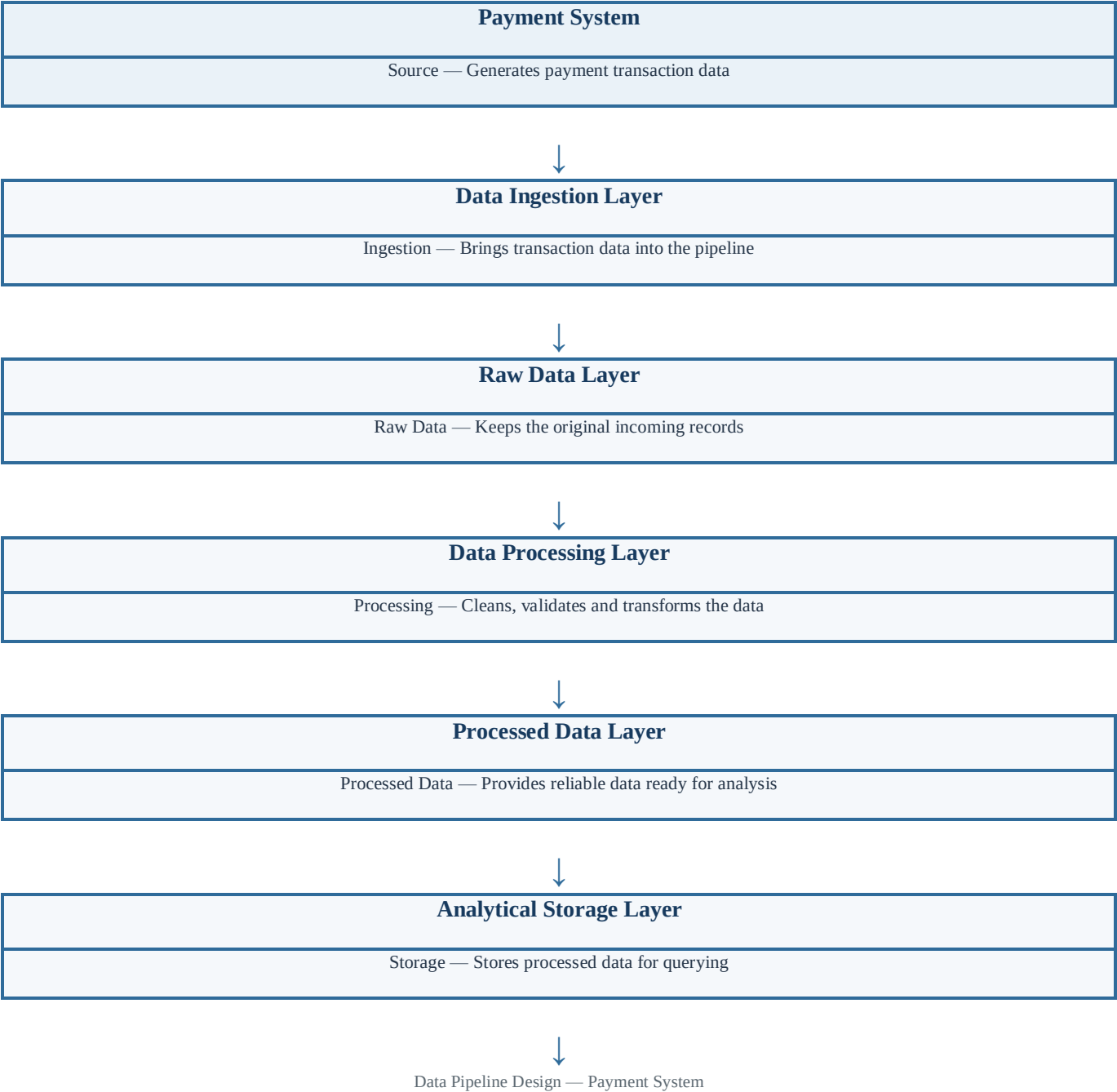
DATA PIPELINE DESIGN

Online Payment System — A Simple End-to-End Payment Data Pipeline

Selected Data Source	Online Payment System
Pipeline Type	Transaction / Periodic Data Collection
Primary Use Case	Payment transaction analysis and dashboard reporting

Diagram 1 — Logical Architecture

This diagram describes the logical responsibilities of the pipeline without using specific cloud or product names.

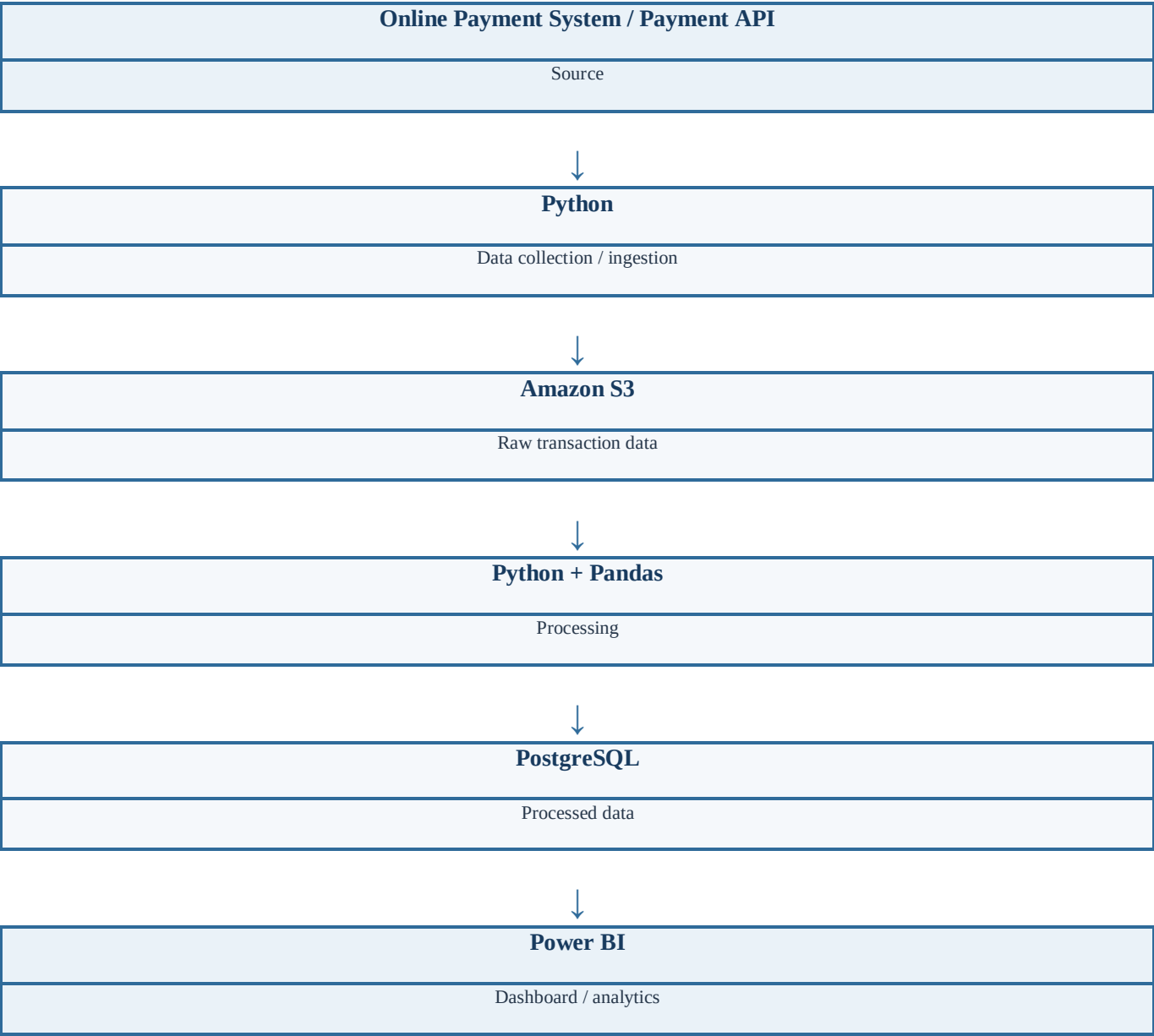


Analytics / Business Users
Consumer — Uses data for monitoring and decisions

Logical flow: Source → Ingestion → Raw Data → Processing → Processed Data → Analytical Storage → Consumer.

Diagram 2 — Technology Mapping

The logical components are mapped to practical technologies. These choices are intentionally simple and are not intended to be the only possible solution.



Technology flow: Payment API → Python → Amazon S3 → Python/Pandas → PostgreSQL → Power BI.

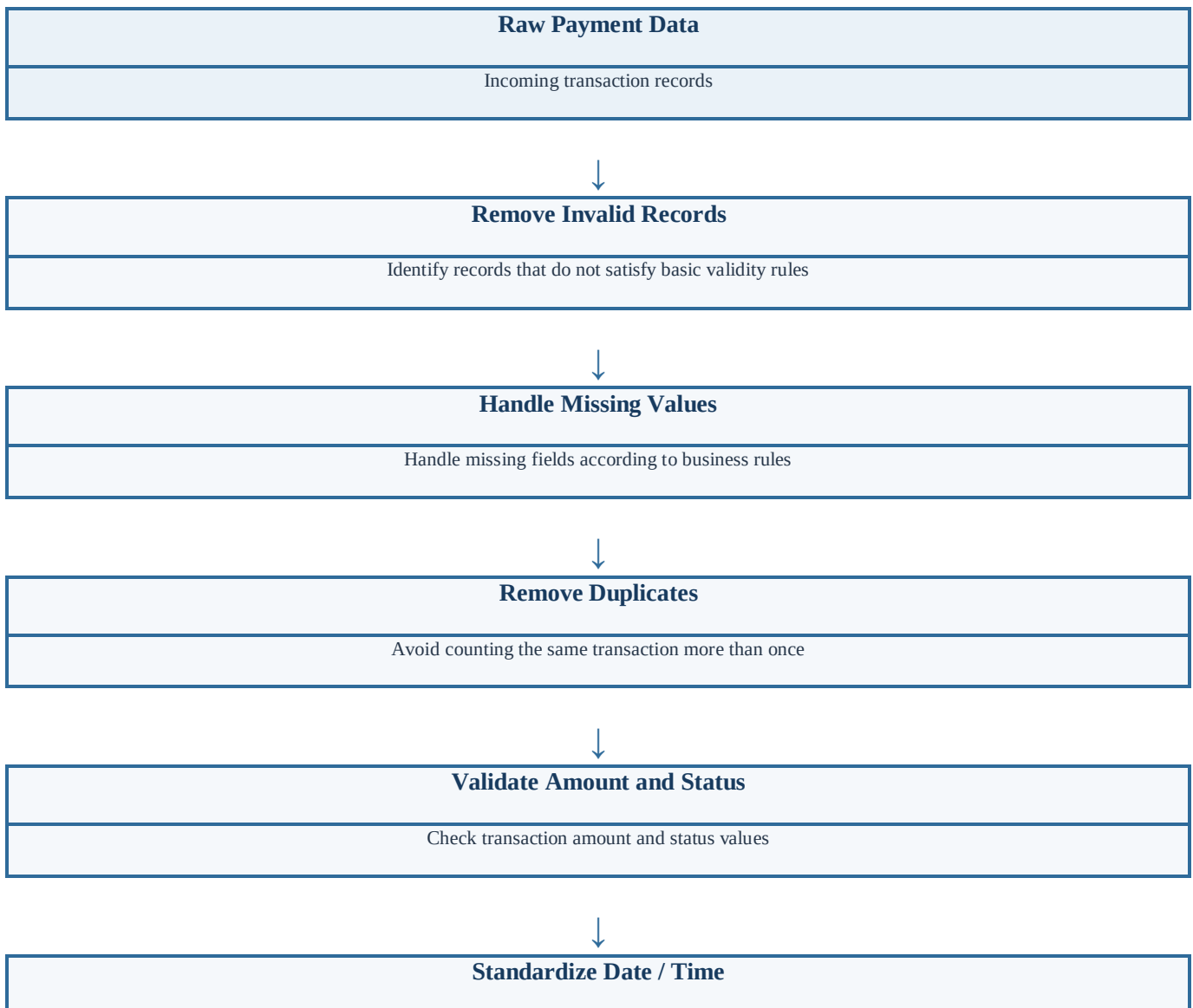
4. Component Explanation

Component	Why it exists
-----------	---------------

Online Payment System / Payment API	Provides the source transaction records generated when customers make payments.
Python	Connects to the source and collects payment transaction data for the pipeline.
Amazon S3	Stores the original raw transaction data so it is preserved before transformation.
Python + Pandas	Cleans, validates and transforms payment records into a consistent structured format.
PostgreSQL	Stores processed payment records in structured tables for efficient querying.
Power BI	Uses processed payment data to create dashboards and reports for users and analysts.

5. Processing Stage

The processing stage converts raw payment transaction records into clean and useful analytical data.



Convert timestamps into a consistent format



Processed Payment Data

Clean, consistent data ready for storage and analysis

Example: A payment amount received as text can be converted to a numeric value. Duplicate transaction records can be removed before the final data is stored.

Short Architecture Explanation

The pipeline starts with the Online Payment System, which is the source of transaction data. A Python program collects the payment records and sends them into the data pipeline. The collected data is first preserved as raw data in Amazon S3 so the original records are available for reprocessing.

The raw transaction data is then processed using Python and Pandas. During processing, invalid records, duplicates, missing values and inconsistent formats are handled. The cleaned data is stored in PostgreSQL, where it can be queried efficiently for reporting and analysis.

Finally, Power BI connects to the processed data and presents dashboards showing information such as transaction volume, successful and failed payments, payment methods and transaction trends.

Why does data move from one component to the next?

Each stage has a clear responsibility: the payment system produces transaction data, Python collects it, S3 preserves the raw data, Pandas prepares it for analysis, PostgreSQL stores structured records, and Power BI presents the results to users. This separation keeps the pipeline simple and understandable.

Question Log — Research & Design Decisions

The following questions arose while designing this specific payment pipeline. They focus on decisions about the payment data flow rather than generic data-engineering questions.

1. Why should the original payment data be preserved before processing?

Because keeping raw records allows the data to be checked or reprocessed if a processing rule changes or an error is discovered.

2. Why can't the dashboard directly read data from the payment system?

The operational payment system is designed to handle transactions, while analytics queries can be separated so they do not interfere with transaction processing.

3. What should happen if the same transaction enters the pipeline twice?

The processing stage should identify duplicate transaction IDs and prevent the same transaction from being counted twice in analytical results.

4. Why do we need to validate payment amount and status?

Invalid amounts or unexpected status values can produce incorrect totals and misleading business reports.

5. Why should dates and times be standardized?

A consistent timestamp format makes filtering, grouping and trend analysis more reliable.

6. Why use separate raw and processed storage?

Raw storage preserves the original records, while processed storage provides cleaner structured data optimized for analysis.

7. Why is PostgreSQL suitable for the processed payment data?

Payment transactions are structured records with relationships and fields that can be efficiently queried using a relational database.

8. Why is a dashboard needed after storing the processed data?

A dashboard converts transaction data into understandable metrics and trends that analysts and business users can monitor.